COURIER PARCEL TRACKING SYSTEM

A CAPSTONE PROJECT REPORT

A Capstone Project Report submitted in partial fulfilment of the requirements for the Course of

Capstone Project

to the award of the degree of

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND ENGINEERING

Submitted by

U. LAKSHIKHAASHRI(1921260155)

Under the Supervision of

Jaisharma Krishnamoorthy

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

SEPTEMBER 2026

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

BONAFIDE CERTIFICATE

This is to certify that the Capstone Project entitled “Courier Parcel Tracking System” has been carried out by U. Lakshikhaashri(1921260155)under the supervision of Jaisharma Krishnamoorthy and is submitted in partial fulfilment of the requirements for the current semester of the B.Tech Computer Science and Engineering program at Saveetha Institute of Medical and Technical Sciences, Chennai.

COURSE COORDINATOR

Dr. Sivagami. S,

Professor

CSE

SIMATS Engineering,

SIMATS, Chennai – 602105.

COURSE FACULTY

Dr. Jaisharma. K,

Professor

CSE / Neural Networks

SIMATS Engineering,

SIMATS, Chennai – 602105.

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

DECLARATION

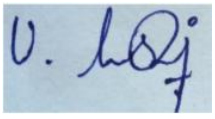
We, U. Lakshikhaashri, S. Aarathana and AYESHA SIDDIQUA.A of the Department of Computer Science and Engineering, SIMATS Engineering, Saveetha Institute of Medical and Technical Sciences, Chennai, hereby declare that the Capstone Project Work entitled “Courier Parcel Tracking System” is the result of our own bonafide efforts. To the best of our knowledge, the work presented herein is original, accurate, and has been carried out in accordance with the principles of engineering ethics and academic integrity. All sources, references, data, and other materials used in the project have been appropriately acknowledged. We further declare that the data, results, analysis, and findings presented in this report have not been fabricated, falsified, or manipulated.

Any external tools, software, datasets, computational resources, or AI-assisted tools used during the project have been appropriately disclosed. We confirm that all applicable ethical, academic, institutional, and professional requirements have been duly followed throughout the planning, execution, analysis, and documentation of the project.

Place: Chennai
Date: _____

Signature of the Students with Name

U.LAKSHIKHAASHRI(1921260155)



SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

Individual Contribution Statement

(Mandatory for all team projects)

Student Name / Register No.	Specific Responsibilities	Design & Development Contribution	Testing & Analysis Contribution	Report Contribution	Approx. Contribution (%)
U. Lakshikhaashri	Hub network topology and IP plan	Designed 3-hub + server topology in Cisco Packet Tracer	Connectivity/ping verification	Network architecture and addressing	33%
S. Aarathana	Parcel status update program	Developed Python TCP server and hub clients	Verified status reception/history	Socket implementation sections	34%
AYESHA SIDDIQUA.A – 1911260141	Wireshark verification and packet-level analysis	Prepared capture configuration and display-filter strategy for TCP port 5000	Captured/analyzed TCP handshake and three status updates; checked ordering/loss	Wireshark methodology, verification, results, analysis, reflection and evidence	33%
Total					100%

Individual Focus: Ayesha Siddiqua.A's contribution begins after the network and application layers are operational. The Packet Tracer topology and Python implementation are team modules; her responsibility was to observe and validate the resulting network traffic using Wireshark.

ABSTRACT

Courier operations depend on timely and reliable status information as a parcel moves from an origin hub to a transit hub and finally to a destination. The project addresses the engineering problem of maintaining an ordered and verifiable parcel-status history across multiple hub clients. A small laboratory model was implemented using Cisco Packet Tracer, Python TCP socket programming and Wireshark.

The network consists of three hub PCs and one central tracking server connected through a switch using the 192.168.1.0/24 private address space. The Python server listens on TCP port 5000 and stores status messages by parcel ID, while each hub sends a message such as P101, Origin Hub; P101, Transit Hub; or P101, Delivered. Wireshark packet capture is used to isolate application traffic with the display filter tcp.port == 5000 and to examine TCP connection establishment and the parcel-status updates.

The project documentation records the updates in the expected order—Origin Hub, Transit Hub and Delivered—with successful TCP acknowledgements and no observed loss in the demonstration. This packet-level evidence complements the server console history and provides an independent verification layer rather than relying only on application output. The verification is limited by the local-network setup, in-memory server history and lack of encryption. Overall, the project demonstrates how packet analysis can be integrated with network simulation and socket programming to validate a practical courier tracking workflow.

Keywords: Courier Tracking, TCP, Wireshark, Packet Capture, Socket Programming, Network Verification.

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

LIST OF ABBREVIATIONS / SYMBOLS

Symbol	Description	Unit / Context
TCP	Transmission Control Protocol	Transport layer
IP	Internet Protocol	Network layer
LAN	Local Area Network	192.168.1.0/24
PC	Personal Computer	Hub client
ACK	Acknowledgement	TCP control flag
SYN	Synchronize sequence numbers	TCP connection setup
IPv4	Internet Protocol Version 4	192.168.1.0/24
Wireshark	Network protocol analyzer	Packet capture
Pkt	Packet Tracer project file	Cisco simulation
P101	Demonstration parcel tracking ID	Synthetic ID
5000	Application/server TCP port	Port number

CHAPTER 1

INTRODUCTION AND ENGINEERING PROBLEM

1.1 Background

Courier tracking is a client–server communication problem in which multiple operational hubs generate status information and a central service maintains the parcel history. In this project, Hub 1 represents the origin stage, Hub 2 represents transit, and Hub 3 represents the destination/delivery stage. Each hub acts as a TCP client and sends a status update to the tracking server. The demonstration parcel P101 progresses from Origin Hub to Transit Hub to Delivered.

1.2 Need for the Project

Without a centralized tracking record, a customer cannot reliably determine the current stage of a parcel and the courier operator has limited visibility into movement between hubs. A networked solution is therefore required to transport updates reliably and maintain an ordered history. Wireshark adds an engineering verification layer by allowing the team to observe exchanged packets rather than relying only on program output.

1.3 Problem Statement

Design and verify a small courier parcel tracking communication system in which three hub clients send ordered status updates to a central server over a local network, while packet-level analysis confirms that the TCP connections and status messages are transmitted successfully and in the intended sequence.

1.4 Project Aim

To demonstrate a reliable and verifiable network-based courier parcel tracking workflow using Cisco Packet Tracer, Python TCP sockets and Wireshark.

1.5 Project Objectives

- Design a three-hub LAN and central tracking-server topology.
- Implement TCP-based hub-to-server parcel status updates.
- Maintain an ordered status history for each parcel ID.
- Capture and filter traffic associated with TCP port 5000.
- Verify TCP connection establishment and status-update sequence using Wireshark.
- Compare application-level results with packet-level evidence.

Objective	Evidence	Verification Role
Reliable status transfer	TCP handshake and ACKs	Inspect TCP packets
Correct order	Three update packets	Check Origin → Transit → Delivered
Server history	Console output	Cross-check with capture
Network operation	Packet Tracer topology	Use topology as capture context

1.6 Scope

Included: three hub clients, one tracking server, switch-based LAN, IPv4 addressing, Python TCP socket communication, server-side parcel history, and Wireshark packet capture and analysis on TCP port 5000. Excluded: public Internet deployment, customer authentication, permanent database storage, GPS tracking, mobile application, encryption/TLS, and concurrent multi-hub server processing. Assumptions include an available laboratory network, a server listening on port 5000, synthetic parcel IDs, and capture only on the team's authorized network. The demonstration operates within the 192.168.1.0/24 LAN using server endpoint 192.168.1.100:5000.

1.7 Expected Engineering Outcome

The expected outcome is a working laboratory prototype integrating network topology, TCP socket communication and packet-level verification. The verification outcome is evidence that hub-generated status updates can be

observed in network traffic and matched to the expected application sequence. The supplied project documentation reports that all three hubs communicate with the server and that Wireshark captures the three status updates in order, with a TCP handshake and no observed loss during the demonstration.

CHAPTER 2

LITERATURE REVIEW AND EXISTING SOLUTIONS

2.1 Review Method

The review was based on computer-networking concepts and the technologies used directly in the project: TCP/IP communication, socket programming, packet capture and LAN simulation. The team documentation references standard networking texts, IETF RFCs, Python socket documentation, Wireshark documentation and Cisco Packet Tracer learning resources.

2.2 Review of Existing Work

Traditional manual tracking relies on individual records or operator updates and does not provide an integrated packet-level proof of delivery. Commercial courier platforms provide richer tracking, but their internal network behavior is not directly observable in a student laboratory. A socket-based prototype provides a controlled environment for studying client–server communication, while Wireshark provides visibility into the protocol exchanges that support the application.

2.3 Comparative Analysis

Existing Approach	Advantages	Limitations	Relevance to Proposed Work
Manual records	Simple	No centralized real-time history; difficult to verify order	Baseline problem
Commercial platform	Rich customer features	Internal packets not available for lab analysis	Real-world inspiration
Python TCP prototype	Low cost; reproducible; clear flow	Local LAN; in-memory; no encryption	Core implementation
TCP + Wireshark	Packet-level evidence and visibility	Requires capture access and protocol knowledge	Verification contribution

2.4 Research / Engineering Gap

The gap addressed is the absence of a simple educational model that connects network design, application-layer socket programming and packet-level verification in one workflow. The verification module checks whether observed TCP traffic supports the claims made by the application output.

2.5 Proposed Contribution

The proposed system combines a simulated LAN, Python TCP communication and Wireshark analysis, making the project inexpensive, reproducible and suitable for demonstrating the relationship between application messages and the packets that carry them.

CHAPTER 3

ENGINEERING DESIGN & TRADE-OFFS, SUSTAINABILITY, ETHICS, SAFETY, RISK & SECURITY

3.1 Requirements

Stakeholder		Requirement
Courier operator / student user		Receive and verify ordered parcel-status updates
Project team		Reproducible laboratory setup with clear module separation
Requirement	**Type (Functional/Non-Functional)**	**Measurable Target**
Capture hub-to-server TCP traffic	Functional	TCP port 5000 visible
Verify connection establishment	Functional	SYN/SYN-ACK/ACK identifiable
Verify parcel update sequence	Functional	Origin → Transit → Delivered
Cross-check application output	Functional	Capture agrees with server history
Reliable communication	Non-Functional	No observed loss in demonstration
Reproducibility	Non-Functional	Same filter/procedure usable again
Authorized analysis	Non-Functional	Only team's lab network

3.2 Constraints & Applicable Standards

Standard / Code	Requirement	How Applied in This Project
RFC 793 – TCP	Connection-oriented reliable transport	Status messages use TCP
RFC 791 – IPv4	IPv4 addressing	192.168.1.0/24
RFC 1918	Private IPv4 space	192.168.1.x addresses
IEEE 802.3 / 802.11	LAN physical/data-link context	Ethernet/Wi-Fi capture
Wireshark	Protocol capture/analysis	Filter and inspect TCP 5000
PEP 8	Python coding conventions	Referenced for client/server code

3.3 Design Specifications

Parameter	Target/Requirement	Unit	Priority
Network	192.168.1.0/24	IPv4 subnet	High
Server	192.168.1.100	IPv4 address	High
Application port	5000	TCP port	High
Parcel test ID	P101	Identifier	Medium
Status sequence	Origin → Transit → Delivered	Ordered states	High
Wireshark filter	tcp.port == 5000	Display filter	High

3.4 Alternative Solutions & Evaluation

Alternative 1 – UDP messaging: Lightweight but lacks TCP's connection-oriented reliability and ordering guarantees required by the project. Alternative 2 – TCP sockets: Provides reliable and ordered transport and remains simple enough for a laboratory prototype. Alternative 3 – HTTP/Web API: Closer to a production service but introduces extra application-layer complexity not required for the networking objective.

Criterion	Weight	UDP	TCP Sockets	HTTP API
Reliability / order	30%	Low	High	High
Implementation simplicity	20%	High	High	Medium
Packet visibility	20%	High	High	High
Project fit	20%	Medium	High	Medium
Extensibility	10%	Medium	Medium	High

3.5 Selected Design & Detailed Design

TCP sockets were selected because the project requires reliable and ordered status delivery. Wireshark was selected because it can expose TCP control packets and application traffic without changing the client/server programs. The Packet Tracer topology provides the communication path, the Python programs create application traffic, and Wireshark observes that traffic at packet level. A topology failure prevents communication; a socket failure prevents status generation; and a capture configuration error can hide valid traffic.

Insert actual Packet Tracer topology screenshot here.

Figure 3.1 – Three-hub tracking network architecture

Insert actual project data-flow diagram/screenshot here.

Figure 3.2 – Parcel status data flow through TCP port 5000

3.6 Trade-off Analysis

Design Decision	Alternative Considered	Benefit	Disadvantage	Final Decision & Justification
Transport	UDP	Low overhead	No reliable ordered delivery	TCP selected for reliable ordered status history
Verification	Console only	Simple	No independent packet inspection	Wireshark selected for packet-level evidence
Storage	Database	Persistent	More setup complexity	In-memory retained for prototype

3.7 Sustainability, Ethics, Safety, Risk & Security

Domain	Consideration	Evidence Evaluated	Impact on Final Decision
Sustainability	Software-only lab setup	Existing computers and free/open educational tools	No dedicated hardware required
Ethics	Protect privacy during capture	Synthetic parcel IDs; own authorized network	Capture limited to lab traffic
Health & Safety	Normal computer-lab operation	No hazardous physical components	Standard lab precautions
Security	Prototype is unencrypted	Documentation identifies no encryption	Retained as limitation; TLS as future work

Risk Register

Risk	Likelihood	Severity	Risk Level	Mitigation	Residual Risk
Wrong interface	Medium	Medium	Medium	Verify active interface	Low
Filter too broad/narrow	Medium	Medium	Medium	Use tcp.port == 5000	Low
Packets missed	Medium	High	High	Start capture before updates	Medium
Wrong order interpretation	Low	High	Medium	Cross-check server history	Low
Unencrypted traffic	Low	High	Medium	Use synthetic data; recommend TLS	Low

CHAPTER 4

IMPLEMENTATION, TESTING & RESULTS, PROJECT MANAGEMENT

4.1 Development Approach & Resources

The project follows a modular approach. The network topology is established first, the server and hub clients are executed, and packet capture is performed while status updates are transmitted. Capture is configured before the clients send test messages so that TCP setup and application traffic can both be observed.

4.2 Hardware / Software / Fabrication Implementation & Modern Tools Used

Tool / Software / Equipment	Purpose	Stage Used
Cisco Packet Tracer	Hub/server topology and IP addressing	Network design
Python 3	TCP client/server status communication	Implementation
Wireshark	Packet capture and protocol analysis	Verification
VS Code	Code development/testing	Implementation
Computer/Laptop	Runs tools and captures traffic	All stages
LAN/Ethernet or Wi-Fi	Communication path	Testing
Insert actual Wireshark/implementation screenshot here.		

Figure 4.1 – Packet-level verification workflow

4.3 Test Plan & Verification

Verification procedure: start the tracking server on TCP port 5000; start Wireshark on the active Ethernet/Wi-Fi interface; begin capture before running the hub clients; apply the display filter tcp.port == 5000; observe SYN/SYN-ACK/ACK connection-establishment packets; inspect P101 status traffic; check the order Origin Hub → Transit Hub → Delivered; compare packet evidence with server console history; and save the capture evidence.

Requirement	Test Method	Acceptance Criterion
Server reachable	Run hub client	Connection established
TCP setup visible	Capture server traffic	SYN/SYN-ACK/ACK identifiable
Correct filter	Apply tcp.port == 5000	Relevant server traffic isolated
Origin update	Hub 1 sends P101, Origin Hub	Visible in sequence
Transit update	Hub 2 sends P101, Transit Hub	Follows origin
Delivered update	Hub 3 sends P101, Delivered	Follows transit
No observed loss	Compare capture/history	All updates accounted for

Specification	Required Value	Achieved Value	Status (Pass/Fail)
TCP connection	Established	Established	PASS
Handshake	Visible	Recorded	PASS
Port filter	5000	Isolated	PASS
Status sequence	Origin → Transit → Delivered	Recorded in order	PASS
Updates accounted for	3/3	3/3 documented	PASS

4.4 Results

The project documentation reports that Module 3 captured traffic on Ethernet/Wi-Fi using the filter tcp.port == 5000. The three status updates were observed in the intended order—Origin, Transit and Delivered—and the TCP handshake was visible. This agrees with the application-level server history for P101.

Verification Item	Observed / Reported Result	Engineering Interpretation
TCP connection	SYN/SYN-ACK handshake observed	Transport connection established
Application port	TCP port 5000 traffic isolated	Correct server endpoint analyzed
Status 1	P101 – Origin Hub	Initial stage transmitted
Status 2	P101 – Transit Hub	Intermediate stage follows origin
Status 3	P101 – Delivered	Final stage completes history
Packet loss	No loss observed	Capture agrees with application record
Insert actual result screenshot/graph here.		

Figure 4.2 – Project result and comparison evidence

4.5 Data Analysis & Engineering Judgment

The verification was treated as a consistency check between three evidence layers: the intended topology and endpoint configuration, the application-level server history, and the packets observed in Wireshark. The strongest result occurs when all three agree. The documented test shows that the server uses TCP port 5000, the clients transmit three P101 updates, and packet capture confirms the TCP setup and update sequence. The observed sequence is important because a courier history is meaningful only when status transitions are ordered. Wireshark does not replace TCP's reliability mechanism; it provides evidence that the expected TCP communication occurred in the demonstration.

4.6 Comparison with Existing Solutions & Achievement of Objectives

Objective	Evidence	Achievement (Fully/Partially/Not Achieved)
Design connected hub network	Packet Tracer topology and IP plan	Fully Achieved
Develop TCP status messaging	server.py and hub1/2/3.py	Fully Achieved
Maintain parcel history	Server output for P101	Fully Achieved
Verify packet-level delivery	Wireshark filter and documented capture	Fully Achieved
Production-ready tracking	Prototype limitations	Partially Achieved

4.7 Strengths & Limitations

Strengths	Limitations
Simple, reproducible laboratory setup	Local LAN only
TCP provides ordered/reliable transport	No persistent database
Wireshark adds independent evidence	No GUI/customer portal
Synthetic data supports privacy	Traffic is unencrypted
Clear module separation	Server processes hubs sequentially

4.8 Project Planning, Roles & Budget

Student	Assigned Responsibility	Actual Contribution
U. Lakshikhaashri	Hub network design	Packet Tracer topology, addressing and connectivity
S. Aarathana	Parcel status update program	Python TCP server and hub clients
Ayesha Siddiqua.A	Wireshark verification	Capture setup, filtering, packet inspection, sequence validation and analysis
Item	**Estimated Cost**	**Actual Cost**
Existing computers/laptops	Existing resource	Existing resource
Cisco Packet Tracer	No additional cost	No additional cost
Python 3	No additional cost	No additional cost
Wireshark	No additional cost	No additional cost

CHAPTER 5

CONCLUSION, FUTURE WORK AND REFLECTION

5.1 Conclusion

The Courier Parcel Tracking System demonstrates a laboratory workflow in which three hub clients communicate with a central tracking server over a LAN. The Python TCP programs generate parcel status messages, the server maintains ordered history, and Wireshark provides packet-level verification. The documented capture confirms TCP setup and the three P101 updates in the expected Origin → Transit → Delivered sequence. The project therefore integrates network design, socket programming and protocol analysis into one verifiable workflow.

5.2 Future Work

- Use MySQL or another persistent database so history survives server restart.
- Add a web/mobile interface for customers.
- Add authentication and role-based access.
- Use GPS-based real-time tracking.
- Add SMS/e-mail notifications.
- Use TLS encryption for communication.
- Support concurrent hub connections using threads or asynchronous networking.

5.3 Professional Learning & Individual Reflection

Area	Learning / Reflection
New Knowledge Acquired	Learned to identify TCP connection establishment, use Wireshark display filters, interpret packet-level communication and relate transport-layer behavior to an application workflow.
Engineering & Professional Skills	Developed packet-analysis, evidence-recording, troubleshooting, technical documentation and cross-checking skills.
Most difficult problem	Distinguishing relevant courier traffic from unrelated traffic; solved using the known TCP port/filter and comparison with server output.
Key engineering decision	Used packet-level verification rather than relying only on console output because independent communication evidence was required.
Independent learning	Strengthened understanding of SYN/SYN-ACK/ACK, TCP acknowledgements, display filters and socket-to-packet relationships.
What would be redesigned	Automate status-transition verification and retain packet metadata, while introducing TLS and persistent storage.

REFERENCES

[1] A. S. Tanenbaum and D. J. Wetherall, Computer Networks, 5th ed., Pearson, 2011.

[2] J. F. Kurose and K. W. Ross, Computer Networking: A Top-Down Approach, 8th ed., Pearson, 2021.

[3] W. R. Stevens, UNIX Network Programming, Vol. 1: The Sockets Networking API, 3rd ed., Addison-Wesley, 2004.

[4] IETF, “Transmission Control Protocol,” RFC 793, 1981.

[5] IETF, “Address Allocation for Private Internets,” RFC 1918, 1996.

[6] Python Software Foundation, “socket — Low-level networking interface,” Python Documentation.

[7] Wireshark Foundation, Wireshark User’s Guide.

[8] Cisco Networking Academy, Cisco Packet Tracer Tutorials.

[9] Project team documentation supplied for Courier Parcel Tracking System, Review 2, 2026.

[10] Supplied Python source files: server.py, hub1.py, hub2.py and hub3.py.

APPENDICES

Appendix A – Detailed Calculations / Verification Procedure

No lengthy numerical derivations are required for this software/network prototype. The verification procedure is: start tracking server on TCP port 5000 → start Wireshark on active interface → begin capture → apply filter tcp.port == 5000 → run Hub 1 (P101, Origin Hub) → run Hub 2 (P101, Transit Hub) → run Hub 3 (P101, Delivered) → inspect SYN/SYN-ACK/ACK → inspect update traffic and compare order with server history → if all three updates are present and ordered, mark PASS → stop and save capture evidence.

Appendix B – Engineering Drawings / Schematics

Insert actual network architecture evidence here.

Figure B.1 – Network architecture evidence from supplied project documentation

Insert actual data-flow evidence here.

Figure B.2 – Data-flow evidence from supplied project documentation

Appendix C – Source Code / Code Repository Information

The supplied source contains a central Python TCP server and three hub clients. The server binds to 0.0.0.0:5000, accepts connections, splits incoming data into parcel ID and status, appends the status to parcel history and returns a success message. The clients connect, send their status and print the server reply. Complete source code may be maintained separately in the approved repository.

Appendix D – Datasheets

Not applicable to this software/network prototype.

Appendix E – Experimental / Raw Data

Packet capture evidence and server console history should be attached where available. The report does not fabricate packet screenshots or a .pcap/.pcapng file.

Appendix F – Ethics / Institutional Approval

The packet analysis is limited to the team's authorized laboratory network and uses synthetic parcel identifiers. No external or unauthorized network traffic is intentionally captured.

Appendix G – Project Meeting / Progress Records

Project progress records may be attached by the team if required by the course faculty.

Appendix H – User/Industry Feedback

Not available in the supplied project report.

Appendix I – Publications / Patents / Competitions / Product Outcomes

Not applicable / not reported in the supplied project report.

Appendix J – Individual Contribution Evidence

Evidence Item	How it Supports Individual Contribution
Wireshark filter tcp.port == 5000	Isolates the tracking application's TCP traffic
TCP handshake observation	Confirms connection establishment used by status updates
Three status updates	Packet-level evidence corresponding to Origin, Transit and Delivered
Server-history cross-check	Confirms application output and packet-level evidence agree
Team result documentation	Records Module 3 as Wireshark Verification and reports three ordered updates with no observed loss

CAPSTONE PROJECT OUTCOME MAPPING SHEET

To be completed by the department/faculty assessor.

Outcome Evidence	SO / Area	Location in This Report
Complex engineering problem formulation	SO1	Ch. 1, Ch. 2
Engineering design under constraints	SO2	Ch. 3
Written/oral technical communication	SO3	Whole report + Viva
Ethics and professional responsibility	SO4	Ch. 3 (3.7)
Teamwork and leadership	SO5	Ch. 4 (4.8)
Experimentation, analysis, engineering judgment	SO6	Ch. 4 (4.3–4.6)
Independent acquisition of new knowledge	SO7	Ch. 5 (5.3)
Standards	SO2	Ch. 3 (3.2)
Sustainability and societal/environmental impact	SO2/SO4	Ch. 3 (3.7)
Risk assessment and mitigation	SO2/SO4	Ch. 3 (3.7)
Security considerations	Relevant SO	Ch. 3 (3.7)
Integrated/system-level engineering	SO1/SO2	Ch. 3 (3.5)
Project planning and management	SO5	Ch. 4 (4.8)
Individual contribution	SO1–SO7	Contribution Statement + Ch. 4 (4.8)